

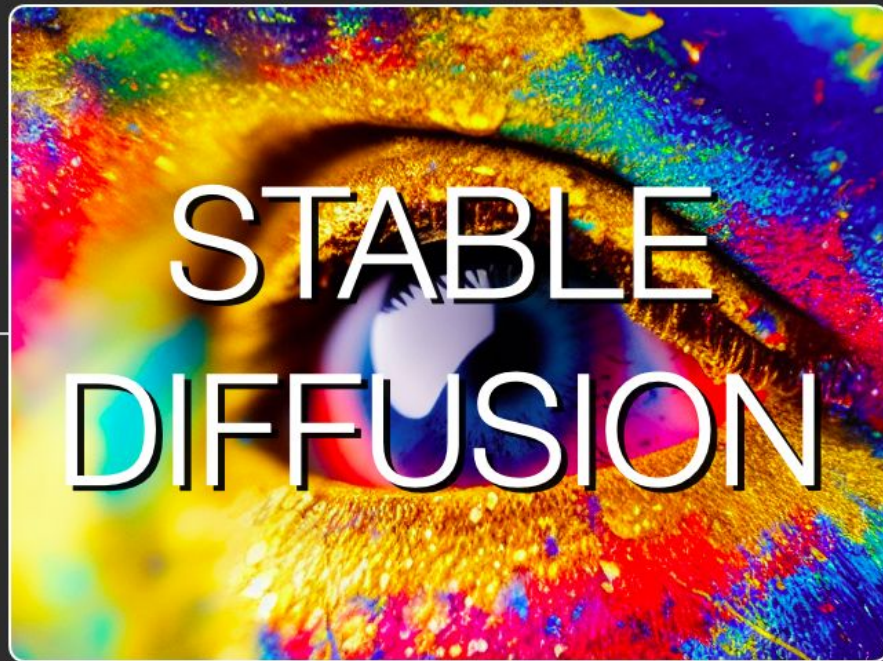
Diffusion Models

Presenters:

Tuan Nguyen, Sean Nian, Tony Hong, Wei Zhang, Yaqi Qiao



The Illustrated Stable Diffusion by Jay Alammarr



Pirate
ship



STABLE
DIFFUSION



Stable Diffusion

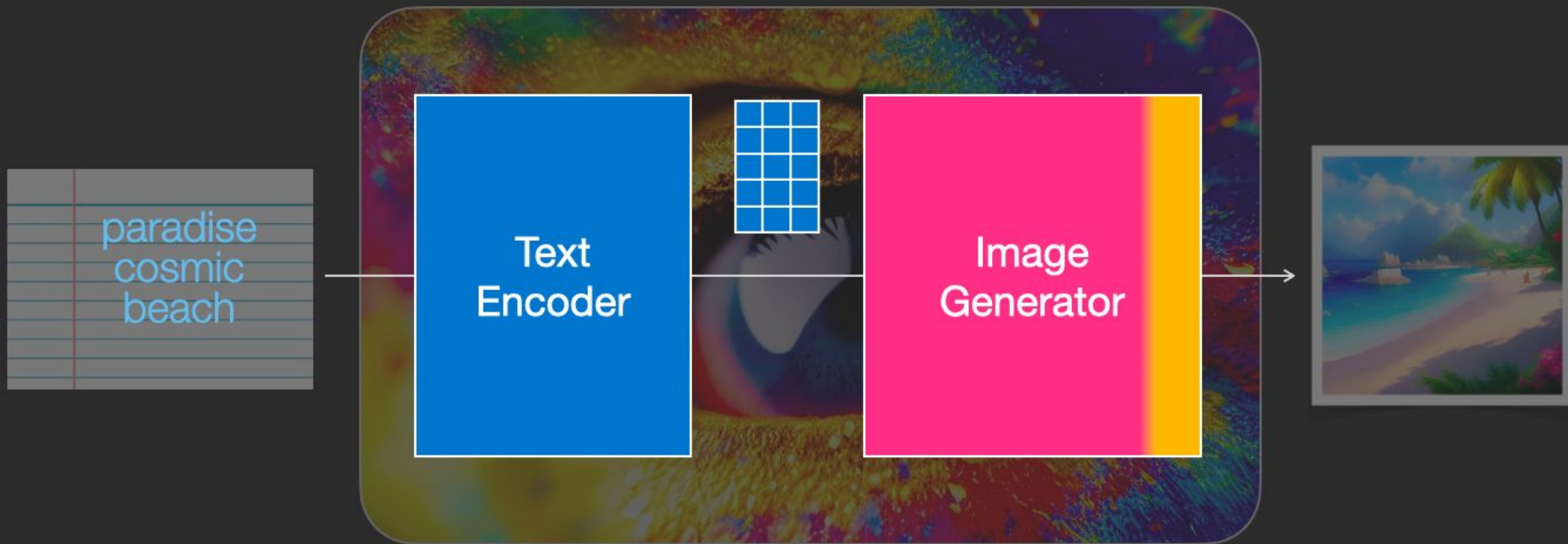
paradise
cosmic
beach

Text
Understander
(Encoder)

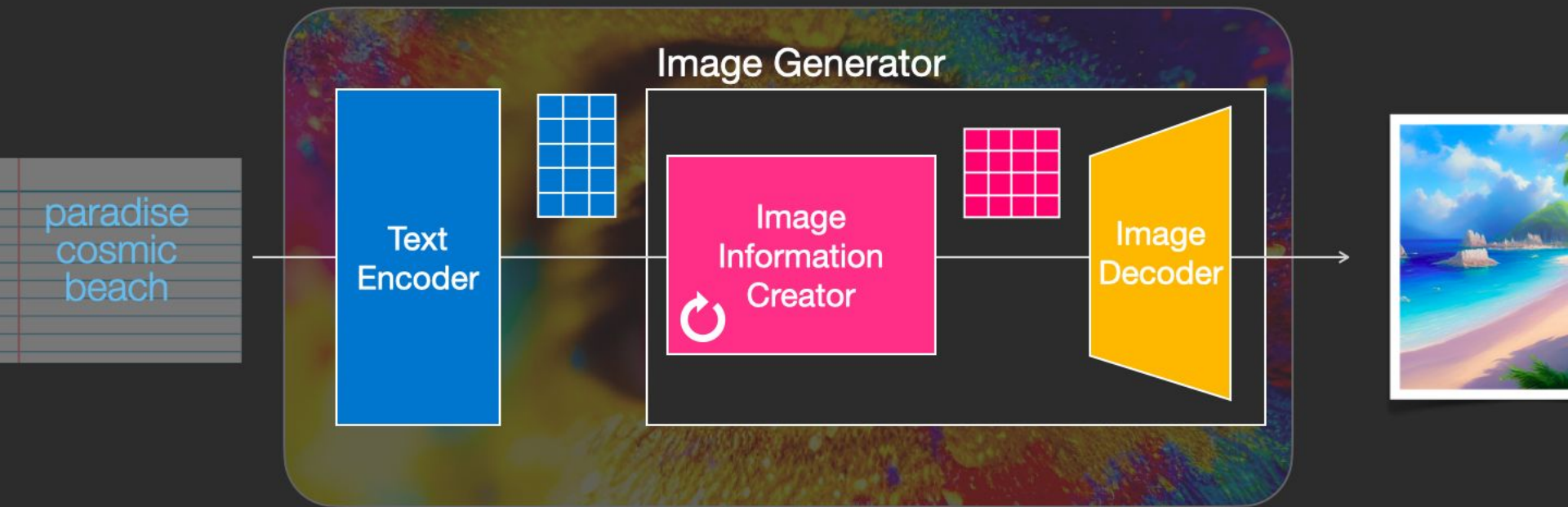
Image
Generator



Stable Diffusion



Stable Diffusion



Input
text

Token
embeddings

Processed image
information tensor

Generated
image

paradise
cosmic
beach

77 tokens

Text
Encoder
(CLIPText)



77 x 768

Image
Information
Creator
(UNet + Scheduler)



4 x 64 x 64

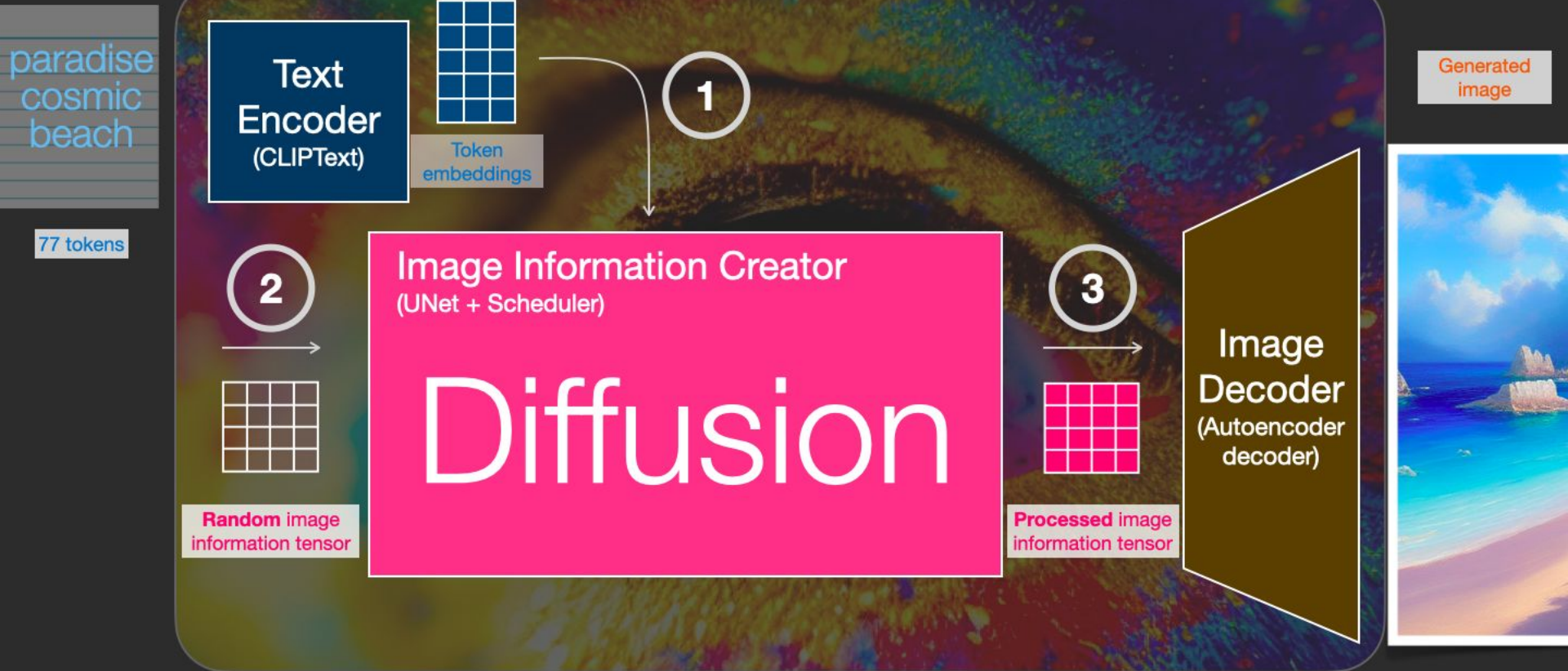
Image
Decoder
(Autoencoder
decoder)



3 x 512 x 512

Stable Diffusion

Stable Diffusion



Stable Diffusion

paradise
cosmic
beach

77 tokens

Text
Encoder
(CLIPText)



Token
embeddings



Random image
information tensor

Image Information Creator
(UNet + Scheduler)

UNet
Step
1



UNet
Step
2



...

UNet
Step
50



Processed image
information tensor

Image
Decoder
(Autoencoder
decoder)

Diffusion

Generated
image



Diffusion



Image Information Creator

Image
Decoder
(Autoencoder
decoder)



Training examples are created by generating **noise** and adding an **amount** of it to the images in the training dataset (forward diffusion)

1

Pick an image



Training examples are created by generating **noise** and adding an **amount** of it to the images in the training dataset (forward diffusion)

1

Pick an image



2

Generate some
random **noise**



Noise sample 1

Training examples are created by generating **noise** and adding an **amount** of it to the images in the training dataset (forward diffusion)

1

Pick an image



2

Generate some
random **noise**

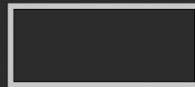


Noise sample 1

3

Pick an amount
of **noise**

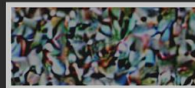
0



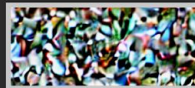
1



2



3



Training examples are created by generating **noise** and adding an **amount** of it to the images in the training dataset (forward diffusion)

1

Pick an image



2

Generate some
random **noise**

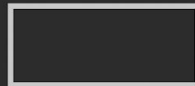


Noise sample 1

3

Pick an amount
of **noise**

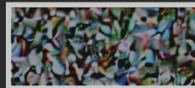
0



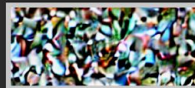
1



2

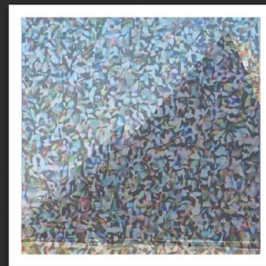


3



4

Add **noise** to
the image in
that **amount**



Generating a 2nd training example with a different image, **noise sample** and **noise amount** (forward diffusion)

1

Pick an image



2

Generate some random noise

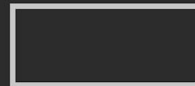


Noise sample 2

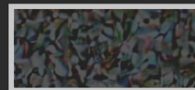
3

Pick an amount of noise

0



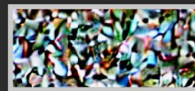
1



2



3



4

Add **noise** to the image in that **amount**



DATASET

MODEL

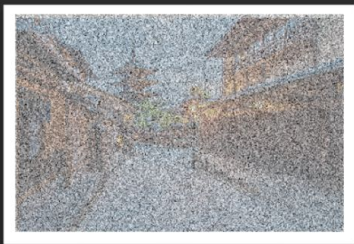
INPUT		OUTPUT / LABEL	
Noise Amount	Noisy Image	Noise sample	
3			
14			
7			
42			
2			
21			

Noise
Predictor
(UNet)

UNet training step

1

Pick a training example
from the training dataset



Noise
amount:
3

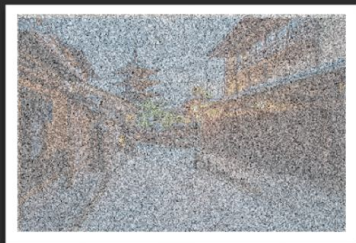
UNet training step

1

Pick a training example
from the training dataset

2

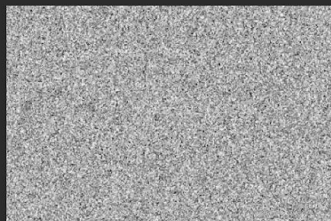
Predict the noise



Noise
amount:
3

UNet

Unet Prediction



UNet training step

1

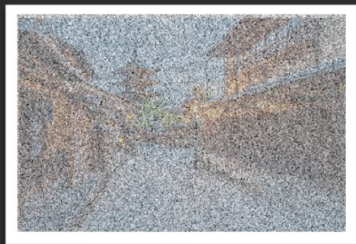
Pick a training example from the training dataset

2

Predict the noise

3

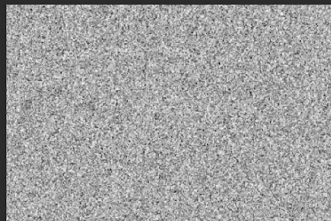
Compare to actual noise (calculate loss)



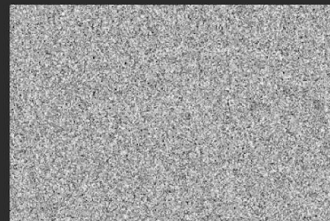
Noise
amount:
3

UNet

Unet Prediction



Actual noise (Label)



UNet training step

1

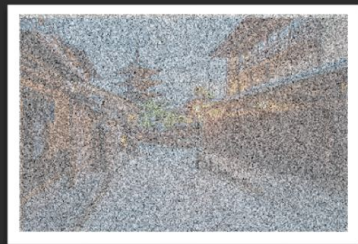
Pick a training example from the training dataset

2

Predict the noise

3

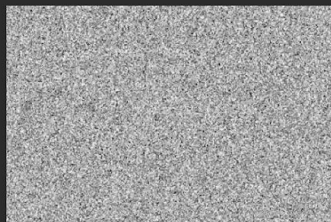
Compare to actual noise (calculate loss)



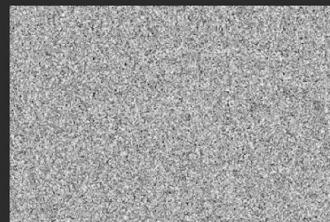
Noise
amount:
3

UNet

Unet Prediction



Actual noise (Label)



4

Update model
(backprop)

Diffusion



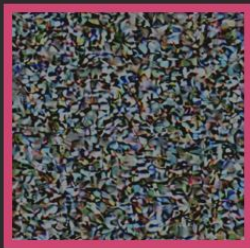
Image Information Creator

Image
Decoder
(Autoencoder
decoder)



Reverse Diffusion (Denoising)

Predicted
noise sample



Noise amount:
3



Trained
Noise
Predictor
(UNet)

Reverse Diffusion (Denoising)

Step 1

Slightly
de-noised
image



=



-
Subtract
predicted noise
from image

Predicted
noise sample

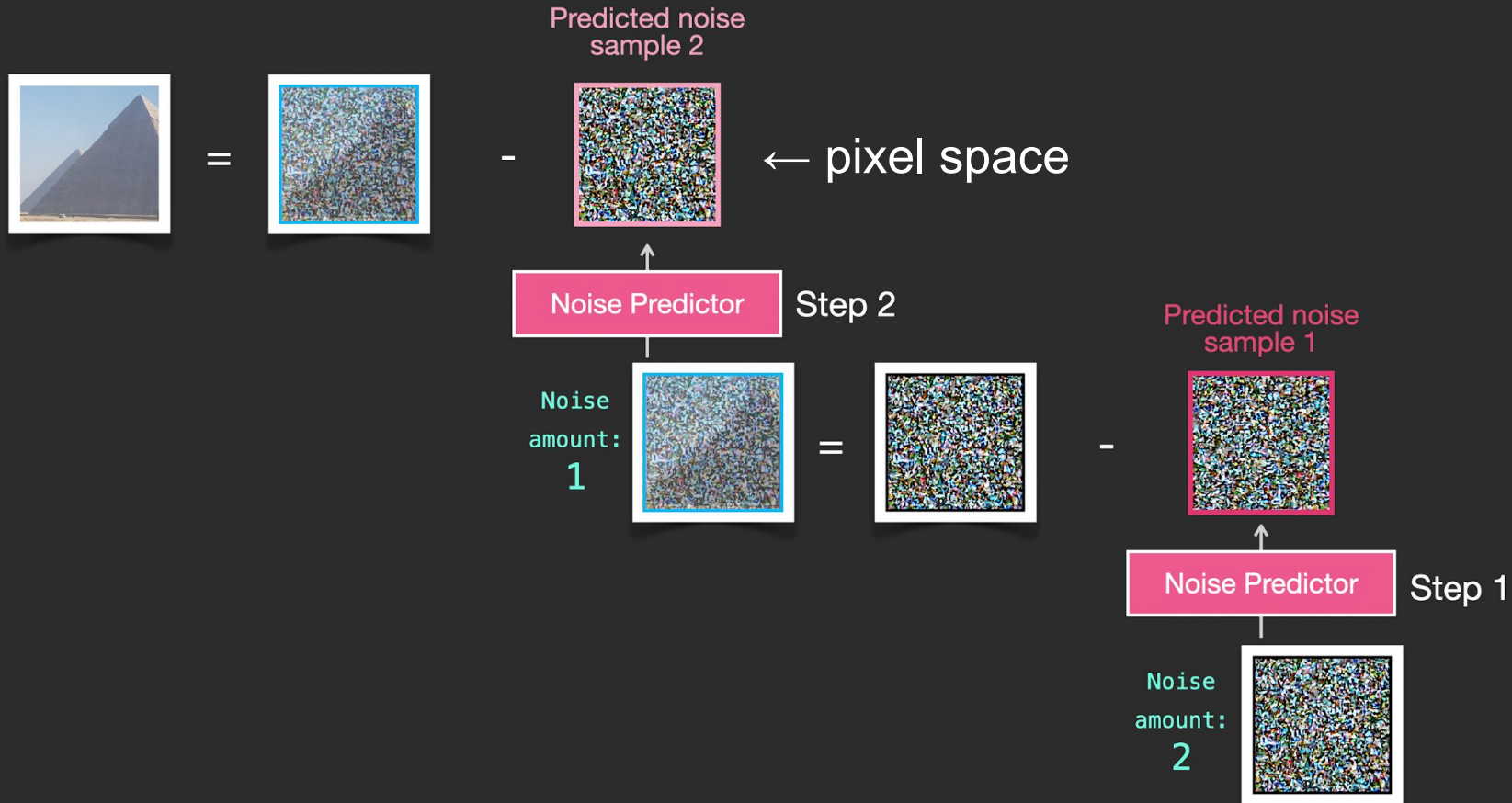


Noise amount:
3

Trained
Noise
Predictor
(UNet)



Image Generation by Reverse Diffusion (Denoising)

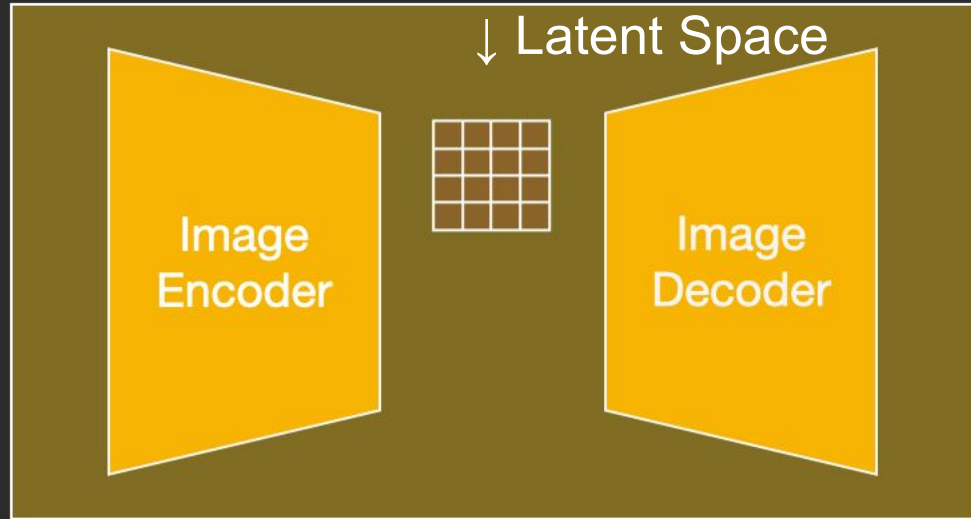


Achieving a Speed Boost: Pixel Compression

Original
image



Generated
image



Original image



Image
Encoder

Generate training examples with different amounts
of noise added to their compressed/latent version



Compressed
image (latent)

Original image

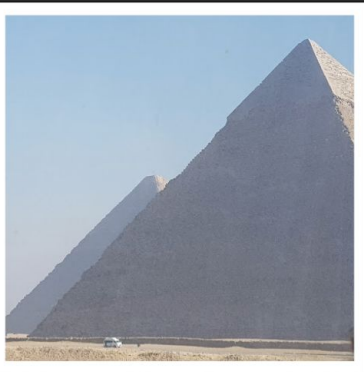


Image
Encoder

Generate training examples with different amounts
of noise added to their compressed/latent version



Compressed
image (latent)



Latent + noise
sample 1 at
noise amount 1



Latent + noise
sample 2 at
noise amount 2



Latent + noise
sample 3 at
noise amount 3

Original image



Image
Encoder

Generate training examples with different amounts
of noise added to their compressed/latent version



Compressed
image (latent)



Latent + noise
sample 1 at
noise amount 1



Latent + noise
sample 2 at
noise amount 2



Latent + noise
sample 3 at
noise amount 3

Image Generation by Reverse Diffusion (Denoising)

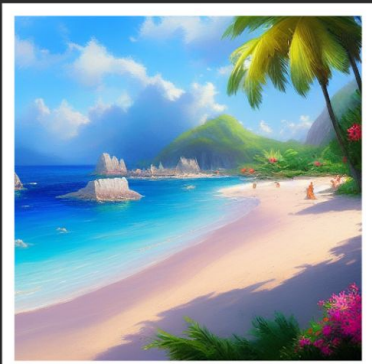


Complete
noise

Image
Decoder

Image Information Creator

Generated image



Original image



Image
Encoder

Generate training examples with different amounts
of noise added to their compressed/latent version



Compressed
image (latent)



Latent + noise
sample 1 at
noise amount 1



Latent + noise
sample 2 at
noise amount 2



Latent + noise
sample 3 at
noise amount 3

Image Generation by Reverse Diffusion (Denoising)



UNet
Step
1



Complete
noise

Image
Decoder

Image Information Creator

Generated image



Original image



Image
Encoder

Generate training examples with different amounts
of noise added to their compressed/latent version



Compressed
image (latent)



Latent + noise
sample 1 at
noise amount 1



Latent + noise
sample 2 at
noise amount 2



Latent + noise
sample 3 at
noise amount 3

Image Generation by Reverse Diffusion (Denoising)



UNet
Step
2



UNet
Step
1



Complete
noise

Image
Decoder

Image Information Creator

Generated image



Original image



Image
Encoder

Generate training examples with different amounts
of noise added to their compressed/latent version



Compressed
image (latent)



Latent + noise
sample 1 at
noise amount 1



Latent + noise
sample 2 at
noise amount 2



Latent + noise
sample 3 at
noise amount 3

Image Generation by Reverse Diffusion (Denoising)



Processed
Image
Information

UNet
Step
50



UNet
Step
2



UNet
Step
1



Complete
noise

Image Information Creator

Image
Decoder



Generated image

How CLIP is Trained

IMAGE



Photo pour Japanese
pagoda and old house in
Kyoto at twilight - image
libre de droit

CAPTION



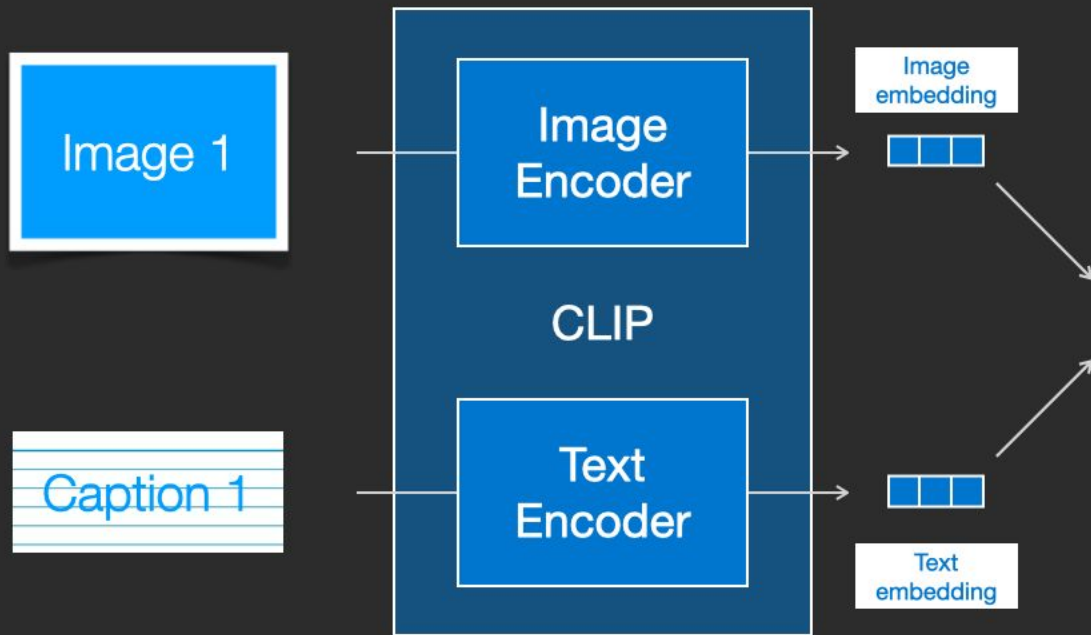
Soaring by Peter
Eades



far cry 4 concept art is the
reason why it 39 s a
beautiful game vg247.
Black Bedroom Furniture
Sets. Home Design Ideas

1

Embed image
and text

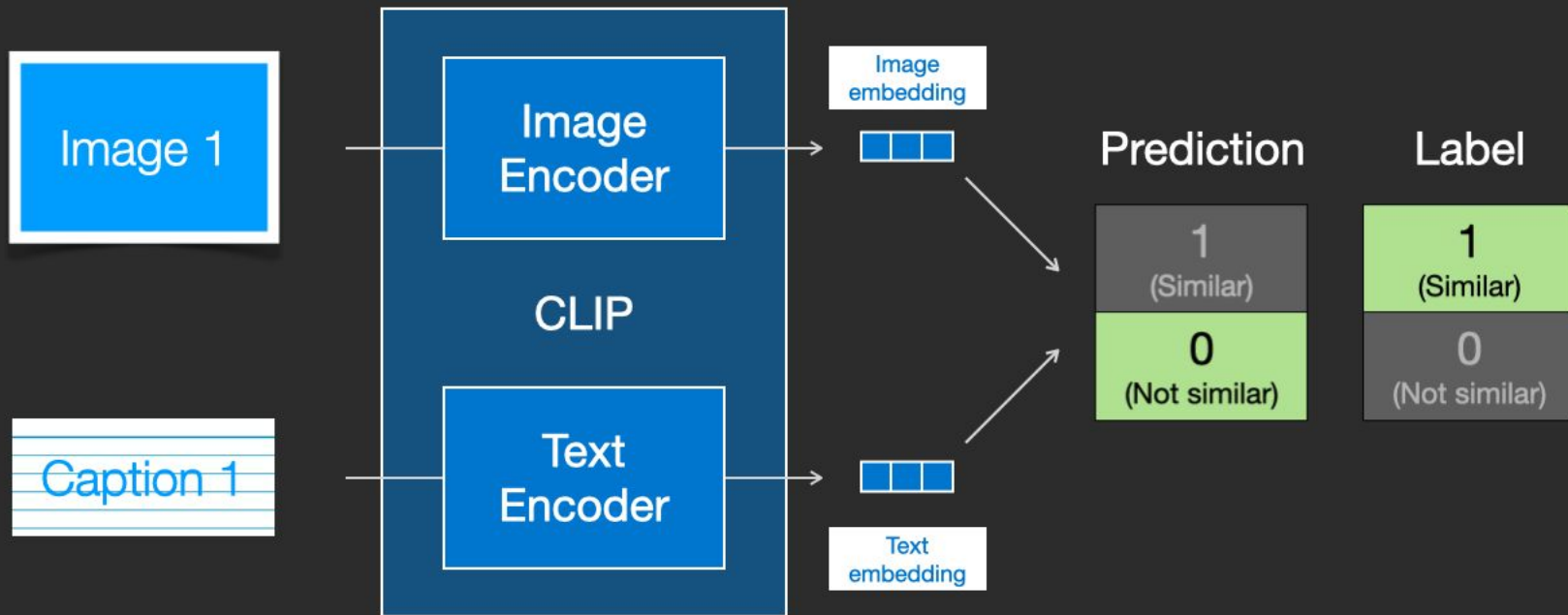


1

Embed image
and text

2

Compare the
embeddings



1

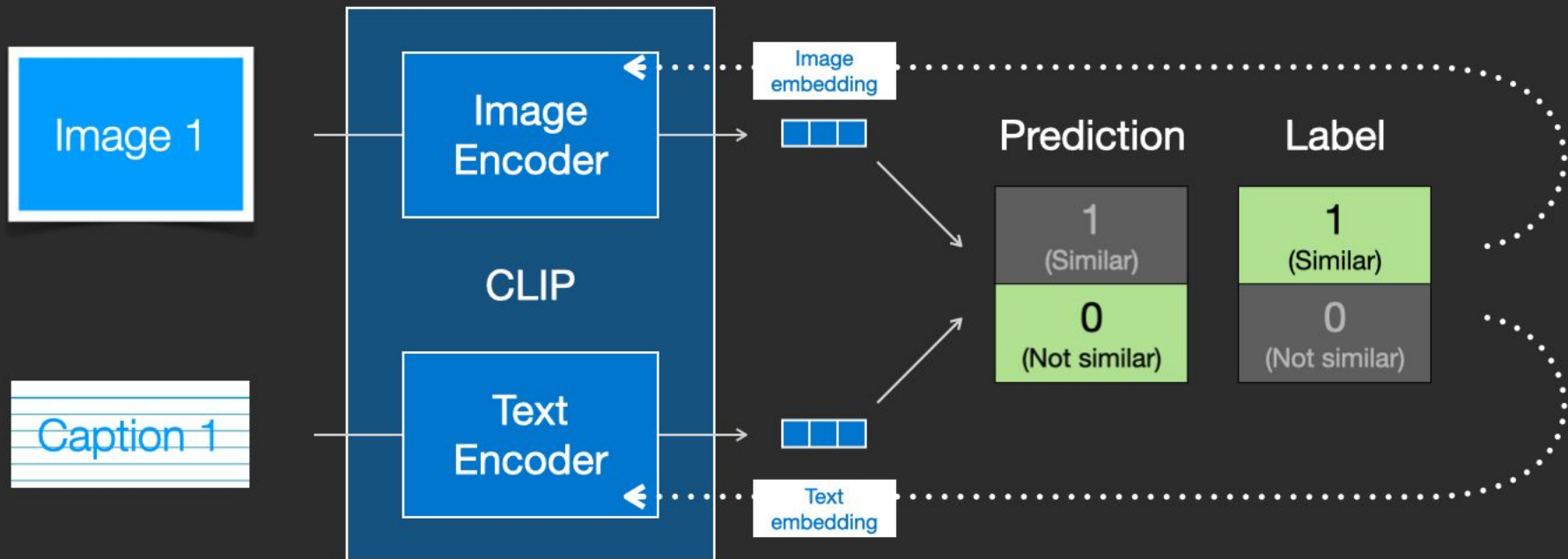
Embed image
and text

2

Compare the
embeddings

3

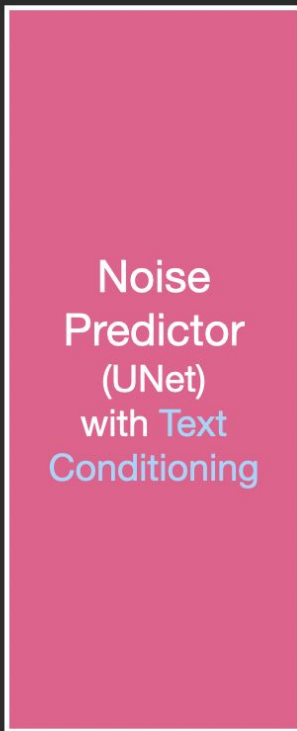
Update the
models





Noise amount:
3

Prompt text
information
(token embeddings)



Predicted noise
sample



DATASET

MODEL

Step	INPUT		OUTPUT / LABEL
	Image	Text	noise sample
3			
14			
7			
42			
2			
21			

Noise Predictor
(UNet)
with Text
Conditioning

Noisy compressed
image
(latents)



Noise amount:
3

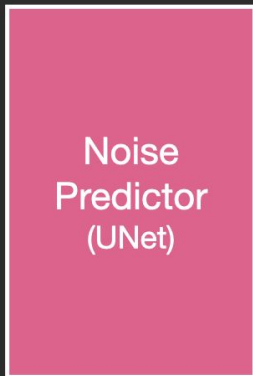
Noisy compressed
image
(latents)



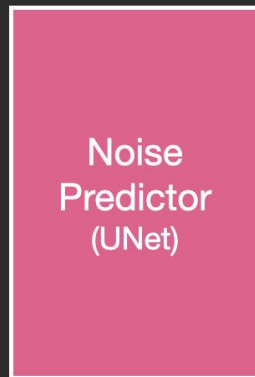
Noise amount:
3



Noise
Predictor
(UNet)



Noisy compressed
image
(latents)



Predicted
noise sample

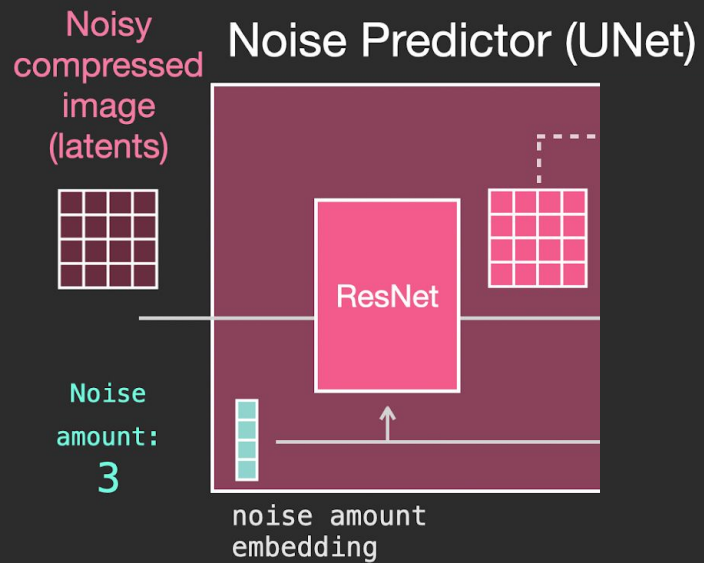
Noise amount:
3



Noisy
compressed
image
(latents)



Noise
amount:
3

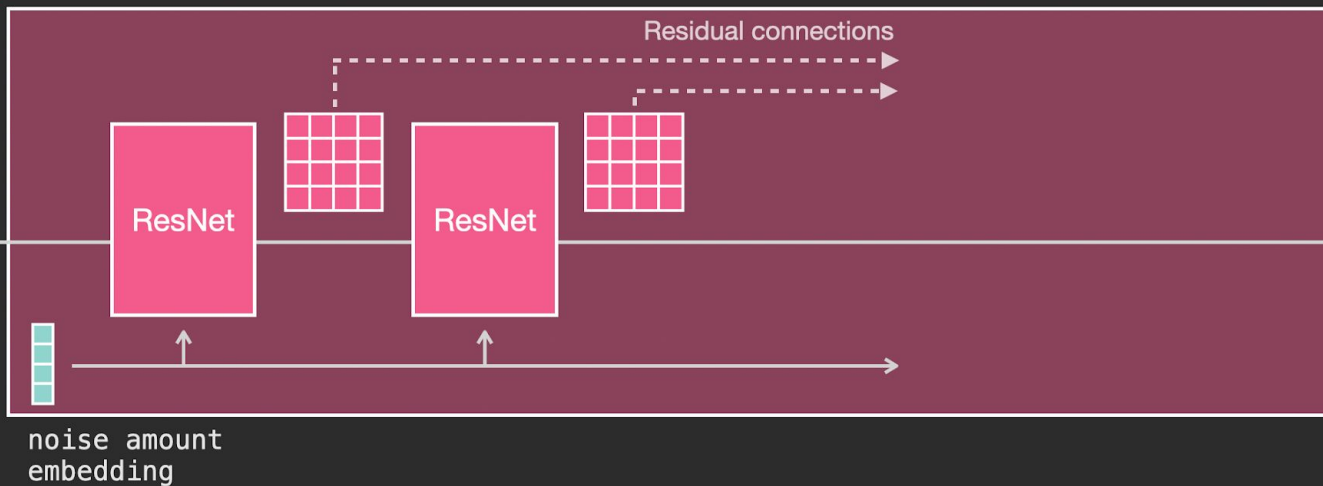


Noisy
compressed
image
(latents)



Noise
amount:
3

Noise Predictor (UNet)

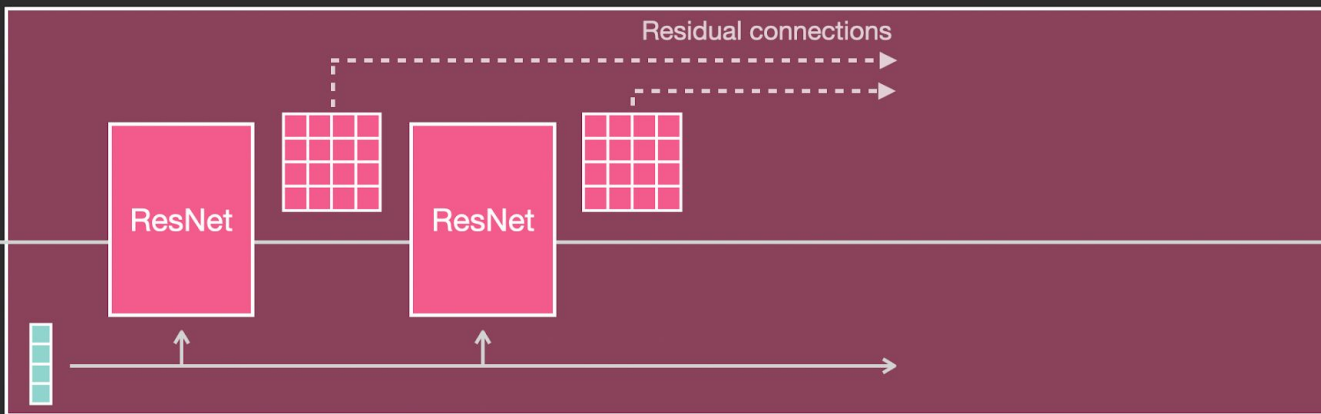


Noisy
compressed
image
(latents)



Noise
amount:
3

Noise Predictor (UNet)



noise amount
embedding

Predicted
noise sample



Noisy compressed
image
(latents)



Text information
(token embeddings)



Noise amount:
3

Noise
Predictor
(UNet)

Predicted
noise sample



Noisy
compressed
image
(latents)

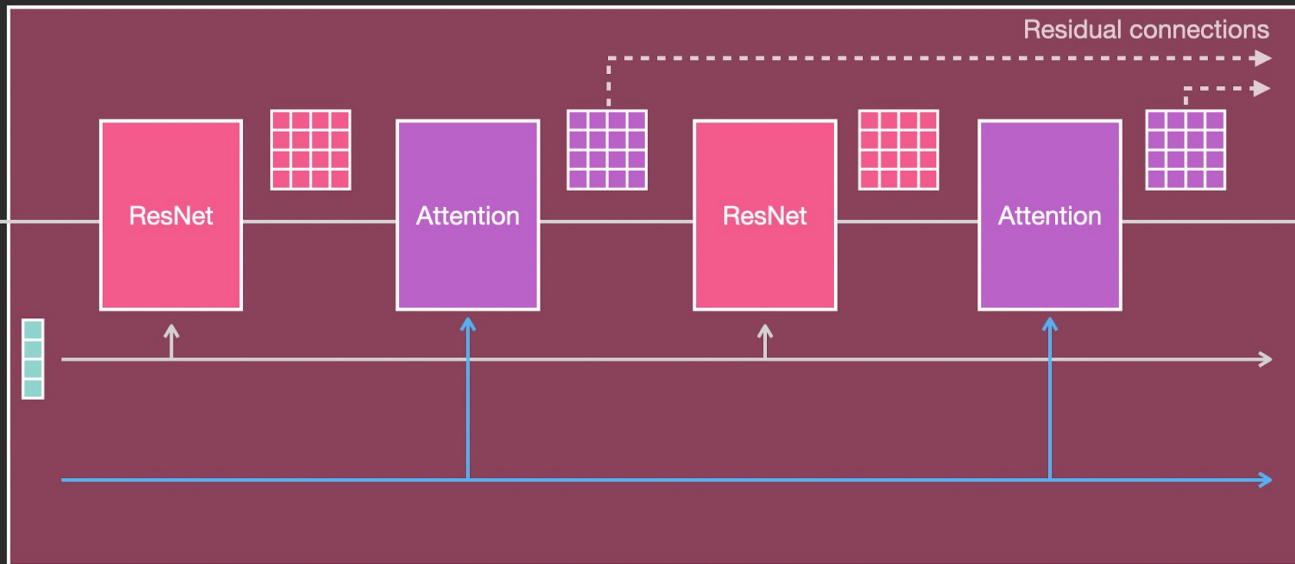


Noise amount:
3



Text information
(token embeddings)

Noise Predictor with Text Conditioning (UNet with attention)



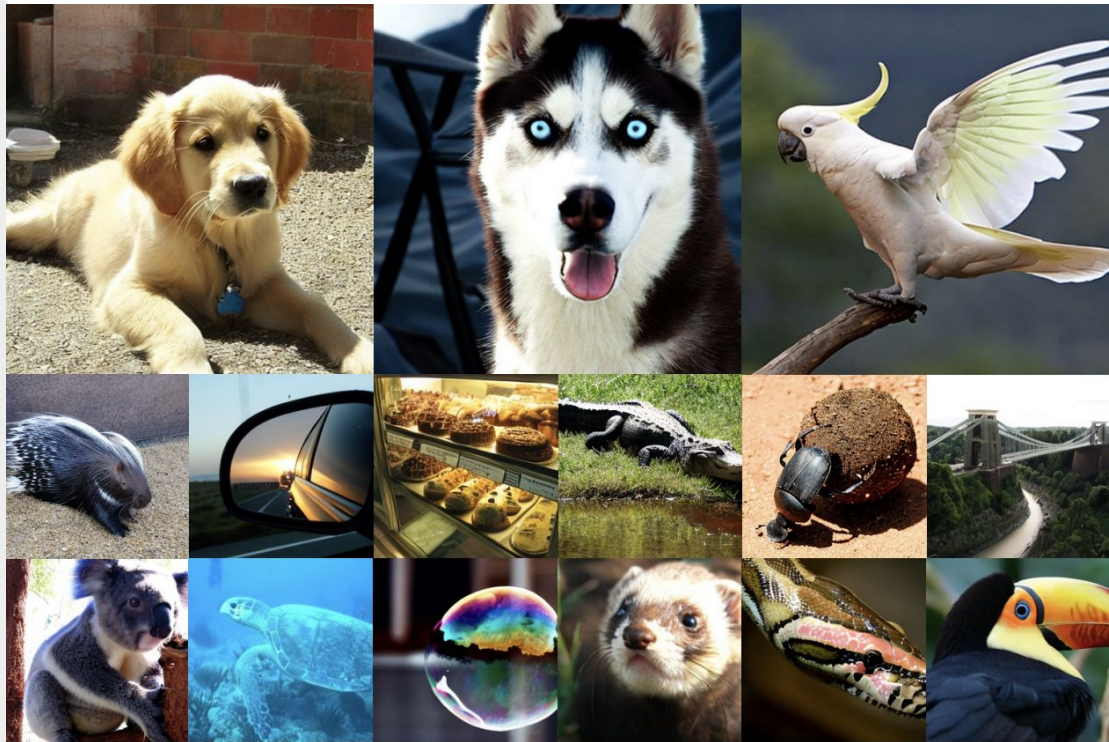
Predicted
noise sample



Scalable Diffusion Models with Transformers

William Peebles
UC Berkeley

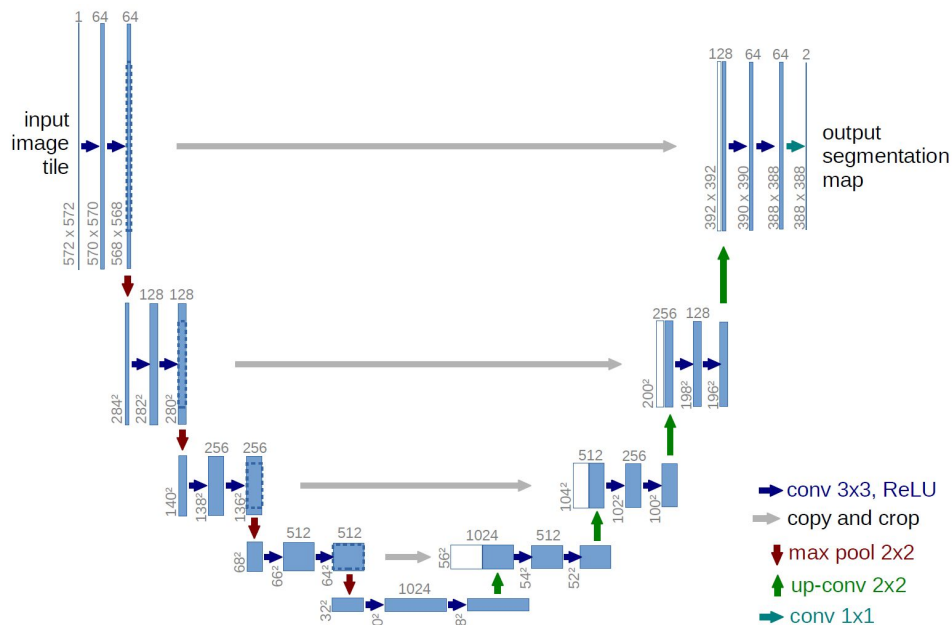
Saining Xie
New York University



From UNet to DiT

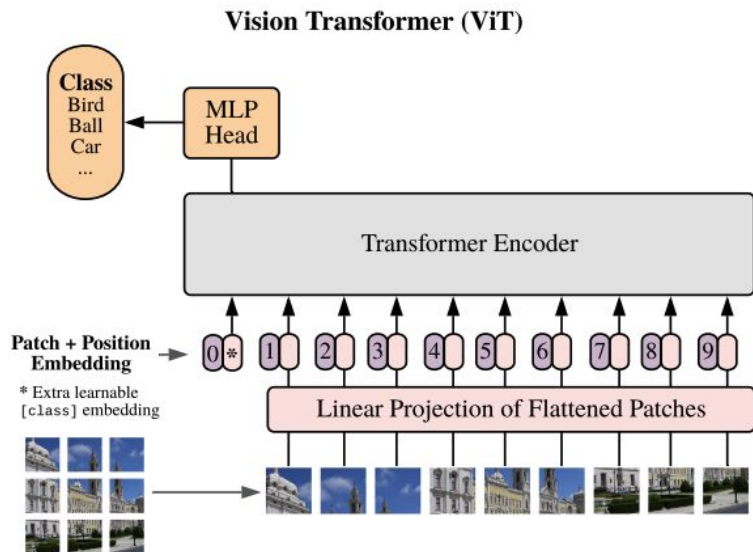
Model	Architecture	Year	Model	Architecture	Year
DDPM	UNet	2020	DiT	DiT	2022
DDIM	UNet	2020	PixArt	DiT	2023
ControlNet	UNet	2023	Sora	DiT	2024
Stable Diffusion v1	UNet	2022	Stable Diffusion v3	DiT	2024

From UNet to DiT

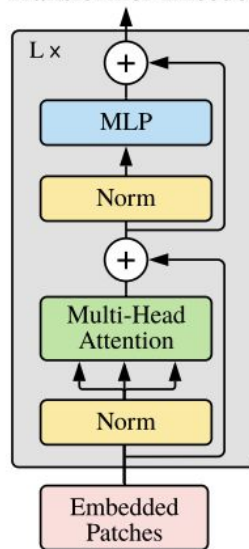


1. Is the inductive bias of convolutions essential for generative modeling?
2. What is the underlying bottleneck for convolution?
3. Is the input size of image fixed for UNet?

From UNet to DiT



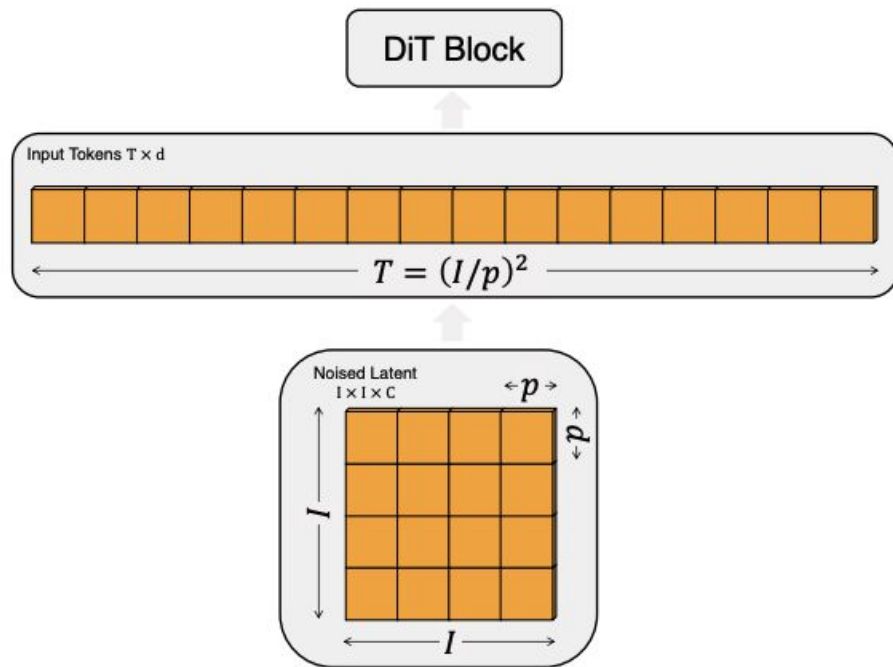
Transformer Encoder



Back to 2021: Transformer architecture has become the standard for natural language processing tasks.

Does it apply to computer vision area?

DiT Architecture: Patchify



Input: Latent z (e.g., $32 \times 32 \times 4$).

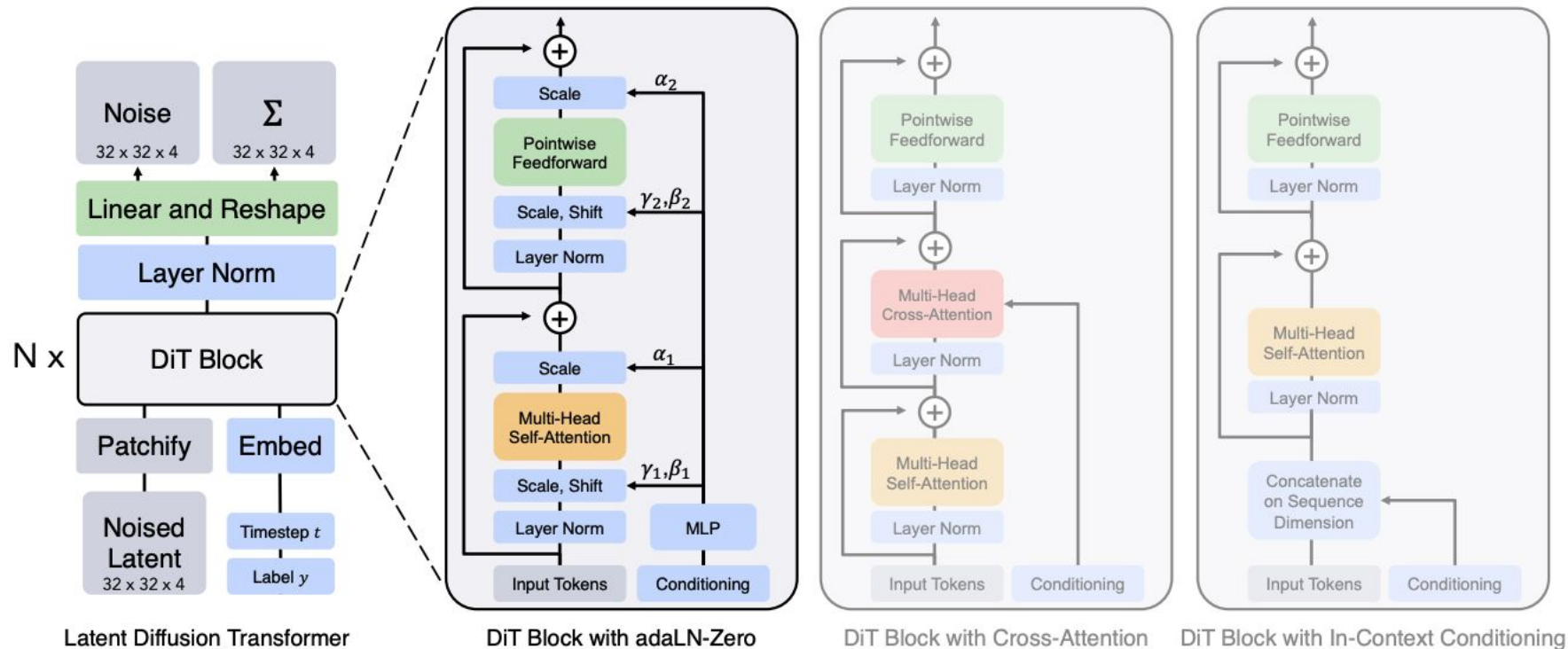
Patchify: Convert the latent spatial grid into a sequence of tokens.

Hyperparameter p : Patch size (e.g., 2, 4, 8). Smaller p means more tokens and more Gflops.

Tokens: $\text{square}(I/p)$.

Larger p leads to fewer tokens

DiT Architecture: DiT Blocks



DiT Architecture: DiT Blocks

Conditioning Strategies:

- In-context: Append t and c as additional tokens.
- Cross-attention: Use a separate layer for conditioning (like ControlNet).
- adaLN (Adaptive Layer Norm): Regress scale and shift parameters from t and c .
- adaLN-Zero (Winner): Initializes the residual block as an identity function for faster convergence.

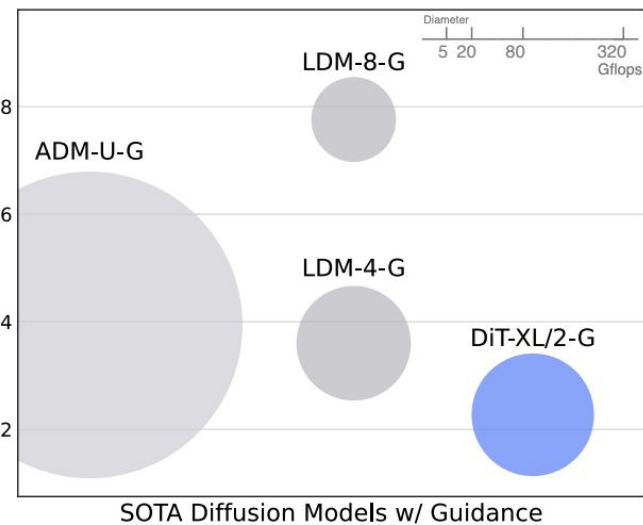
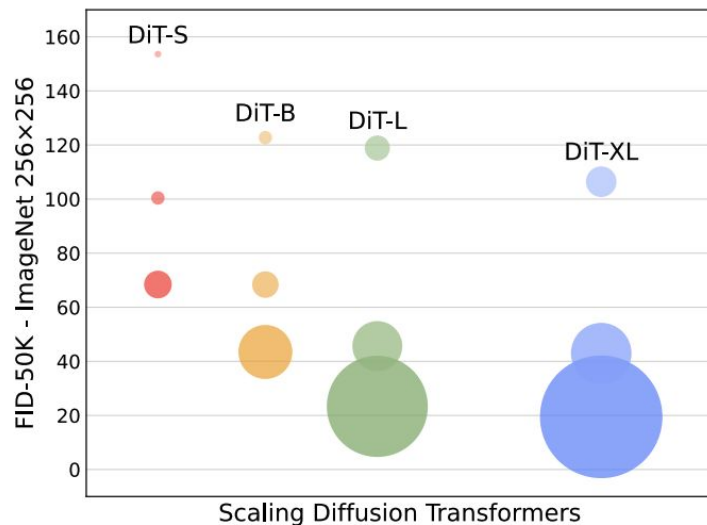
$$\text{adaLN}(h, y) = \gamma(y) \cdot \text{LayerNorm}(h) + \beta(y)$$

$$\text{Output} = \mathbf{x} + \alpha_1 \cdot \text{SelfAttn}(\text{adaLN}(\mathbf{x}, y))$$

$$\text{Final Output} = \text{Output} + \alpha_2 \cdot \text{MLP}(\text{adaLN}(\text{Output}, y))$$

DiT Architecture: Scalability

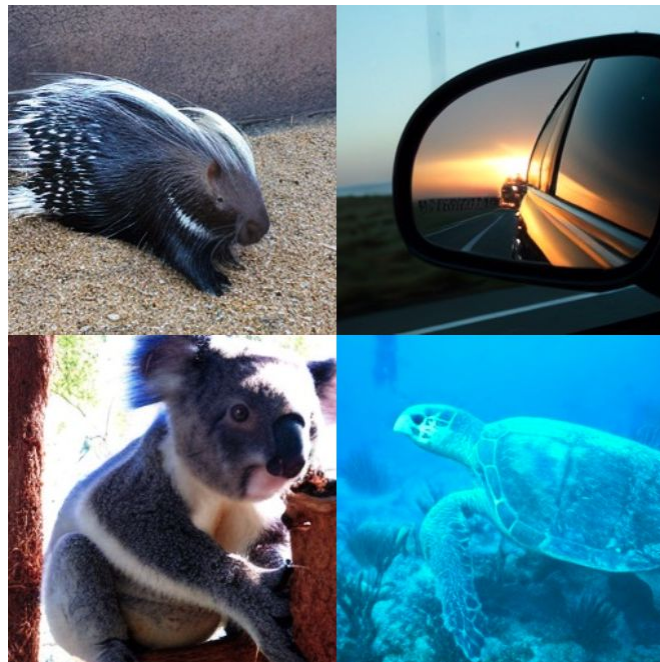
Model	Layers N	Hidden size d	Heads	Gflops ($I=32, p=4$)
DiT-S	12	384	6	1.4
DiT-B	12	768	12	5.6
DiT-L	24	1024	16	19.7
DiT-XL	28	1152	16	29.1



DiT Evaluation

Experimental Setup:

- Dataset: ImageNet 256×256 & 512×512
- Model parameters:
4 sizes (S/B/L/XL) × 3 patch sizes (8/4/2)
- Metric: FID-50K (↓ is better)



Eva0: DiT(-XL/2) Achieves SOTA Performance

Evaluation Results:

- Best FID among all generative models (GANs + diffusion)
- 5.4× more compute-efficient than U-Net models at 512×512

Class-Conditional ImageNet 512×512						Gflops
Model	FID↓	sFID↓	IS↑	Precision↑	Recall↑	
BigGAN-deep [2]	8.43	8.13	177.90	0.88	0.29	2813
StyleGAN-XL [53]	2.41	4.06	267.75	0.77	0.52	
ADM [9]	23.24	10.19	58.06	0.73	0.60	
ADM-U	9.96	5.62	121.78	0.75	0.64	
ADM-G	7.72	6.57	172.71	0.87	0.42	
ADM-G, ADM-U	3.85	5.86	221.72	0.84	0.53	525
DiT-XL/2	12.03	7.12	105.25	0.75	0.64	
DiT-XL/2-G (cfg=1.25)	4.64	5.77	174.77	0.81	0.57	
DiT-XL/2-G (cfg=1.50)	3.04	5.02	240.82	0.84	0.54	

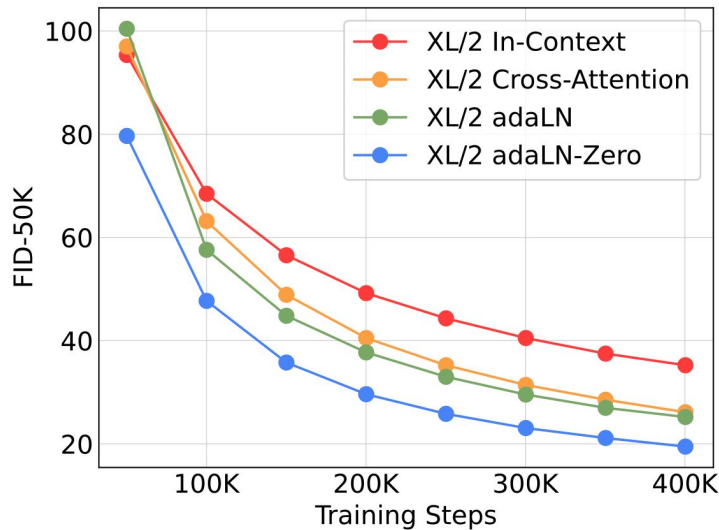
Eva I: AdaLN-Zero is the Optimal Conditioning Design

Evaluation Results:

- Among methods:
In-Context, Cross-Attention,
AdaLN, AdaLN-Zero:
AdaLN-Zero outperforms all others
- **AdaLN-Zero** is the most
compute-efficient

Design Echo:

- Validates the choice of AdaLN-Zero for
injecting timestep/class conditions
- Zero initialization is crucial for training
stability



Gflops (DiT-XL/2):

In-Context – 119.4 Gflops

Cross-Attention – 137.6 Gflops

AdaLN – 118.6 Gflops

AdaLN-Zero – 118.6 Gflops

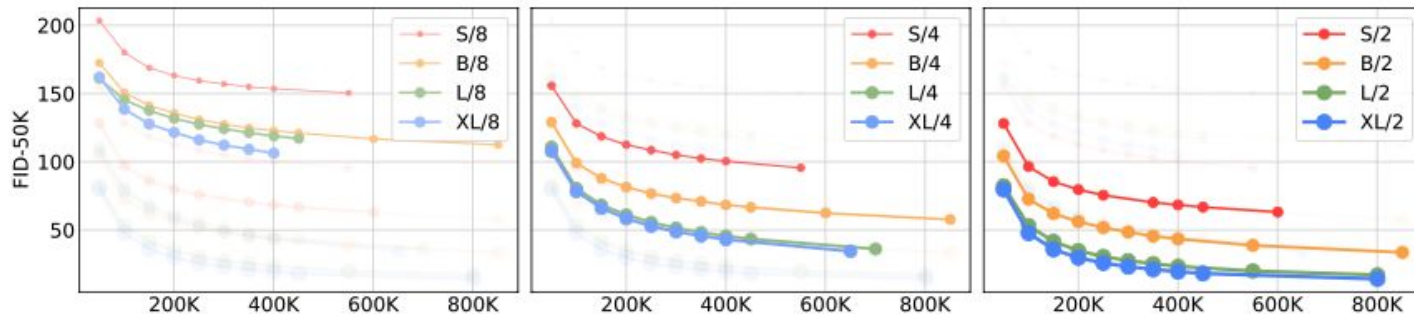
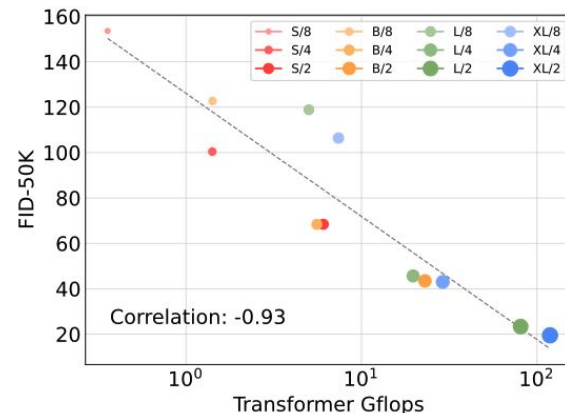
Eva2: DiT Scales Predictably with Model Compute

Scaling along two axes:

- Model size: $S \rightarrow B \rightarrow L \rightarrow XL$ (depth + width)
- Number of tokens: patch size $8 \rightarrow 4 \rightarrow 2$

Key Result:

- Strong correlation: Gflops $\leftrightarrow$ FID = -0.93



Systems for Diffusion Model

Scenario 1: Real-time Interactive Generation

- Video streaming, virtual YouTubers, AR/VR require low latency
- Traditional: Sequential denoising, no streaming support
- Goal: Improve single-GPU throughput

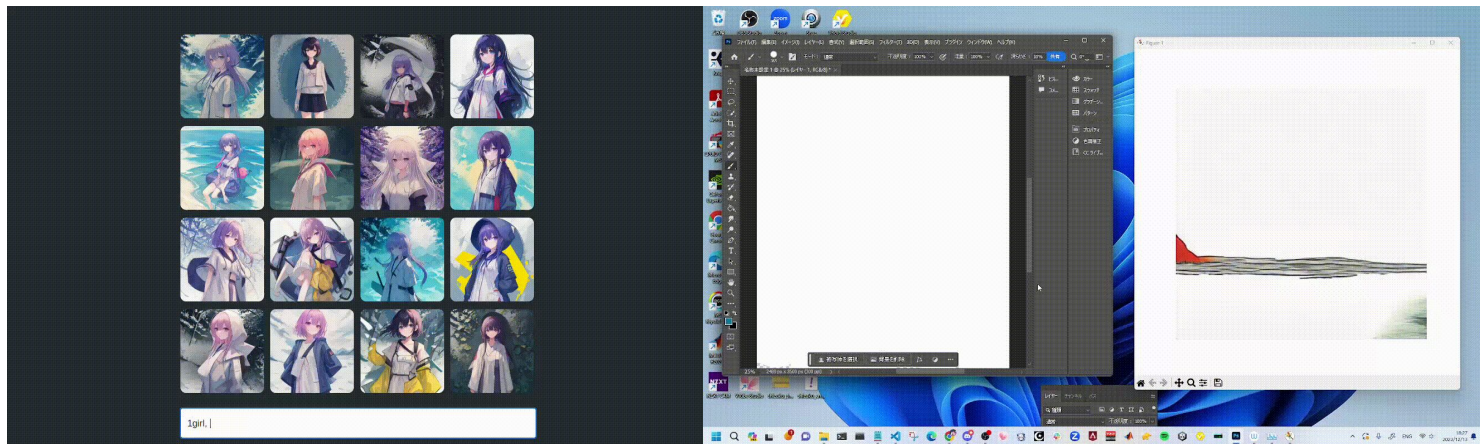
Scenario 2: High-Resolution Image Generation

- Massive computation for 4K images (SDXL: 6,763 GMACs/step)
- Single GPU memory and latency bottleneck
- Goal: Multi-GPU parallel acceleration

StreamDiffusion: Pipeline for Real-time Interaction

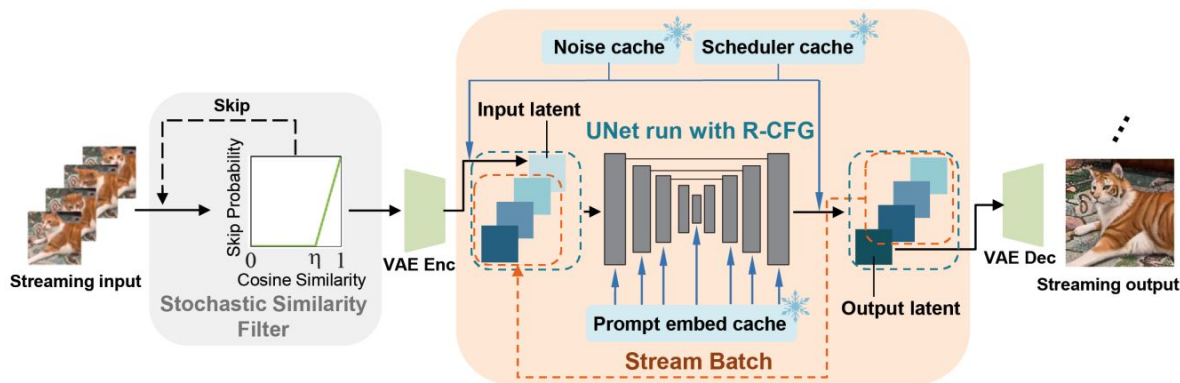
Scenario: Real-time Interactive Generation

Throughput: RTX 4090: 91.07 fps, 59.56× faster than AutoPipeline



StreamDiffusion: Pipeline for Real-time Interaction

- **Stochastic Similarity Filter:** Calculate frame similarity, skip GPU computation for static scenes
- **Stream Batch:** Accept new input after each denoising step instead of waiting and batch staggered denoising steps from different images
- **Residual CFG:** Approximate negative conditions with virtual residual noise, avoiding redundant UNet computations

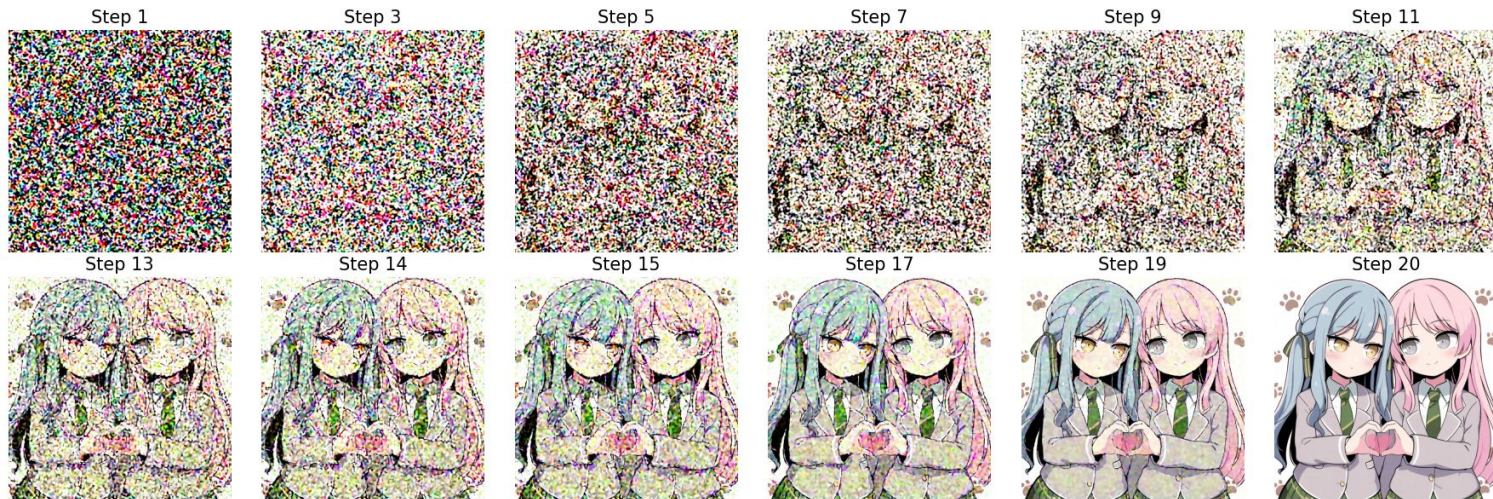


Distributed Parallel Inference

- **Scenario:** High-Resolution + Multi-GPU Dilemma
- Why traditional parallelism for LLMs fails?
 - Only a few billion params → little to split.
 - Huge activation tensors dominate memory bandwidth

DistriFusion

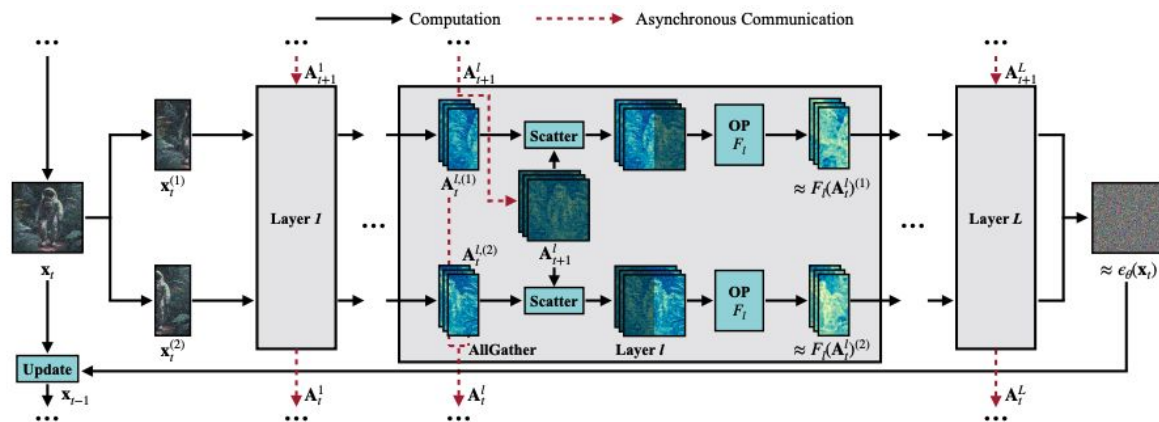
- **Key Observation:** Adjacent diffusion steps have highly similar inputs: Average difference: only **0.02** (within range $[-4, 4]$, **~0.3%**)



DistriFusion

Key Technique: Corrected Asynchronous GroupNorm

- Problem: GroupNorm needs global statistics, synchronous communication costly
- Solution: Correct stale statistics with correction term
- Result: No sync overhead, quality matches synchronous method



PipeFusion

- **vs. DistriFusion:**
 - **DistriFusion:** Only split input patches, full model replicated on each GPU
 - **PipeFusion:** Split both patches and model layers
- **Design:** Partitioning + Micro-step Execution + Exploit Temporal Redundancy
- **Advantages:** Lower Communication Overhead + Higher Memory Efficiency

